WARING DECOMPOSITION OF DERIVATIVES AND ITS APPLICATIONS

SHUIXIN FANG[†] , HANG ZENG[†] , AND TAO ZHOU[†,*]

Abstract. Computing high-order derivatives in neural network residuals is often computationally and memory intensive because automatic differentiation can generate large computational graphs. Within the Taylor-mode framework used by the stochastic Taylor derivative estimator (STDE), we introduce the Waring decomposition of derivatives (WDD) method, which uses power-sum decompositions to represent homogeneous linear combinations of partial derivatives by univariate directional derivatives. For a single mixed partial, a classical roots-of-unity construction gives explicit coefficients and complex directions, and the monomial Waring rank establishes their minimum number within exact linear representations at the derivative order. Experiments on individual high-order partial derivatives show that WDD reduces evaluation time for several tested patterns relative to the deterministic directional Taylor construction reviewed in STDE++. In experiments with physics-informed neural networks (PINNs) for the one-dimensional Korteweg–de Vries (KdV) equation with gradient enhancement, the two-dimensional KdV equation, and the Kadomtsev–Petviashvili II (KP-II) equation, WDD attains comparable mean solution errors in less training time than the compared STDE++ constructions at equal update counts.

Key words. high-order derivatives, Taylor-mode automatic differentiation, Waring decomposition, directional Taylor coefficients

AMS subject classifications. 65D25, 14N07

1. Introduction.

Physics-informed neural networks (PINNs) approximate solutions of partial differential equations (PDEs) by minimizing loss functions constructed from the corresponding PDE residuals. This requires derivatives of the network with respect to its inputs. For high-order equations, the repeated evaluation of these derivatives can dominate both the runtime and memory cost of residual evaluation.

Automatic differentiation is the standard tool for evaluating these derivatives [1, 9, 20]. Several exact approaches reduce its cost by exploiting the structure of the differential operator. For example, forward Laplacian methods propagate the Laplacian through the network without constructing the full Hessian [18]. Such methods can be efficient, but their differentiation rules are designed for specific structured operators and do not readily extend to general high-order partial derivatives. Taylor-mode automatic differentiation provides a more general alternative by propagating a truncated univariate Taylor series through the network [3, 9]. Existing general constructions recover multivariate high-order derivatives from multiple Taylor expansions, often by reconstructing the full derivative tensor [8]. The number of tensor components grows rapidly with both the input dimension and the derivative order, resulting in substantial computational and GPU-memory costs [24].

Randomization offers another route to reducing derivative costs. Stochastic dimension gradient descent uses coordinate sampling to evaluate only a subset of derivative terms during each optimization step [13]. Hutchinson-type estimators use random projections to approximate trace-type differential operators [11,14,21]. However, both approaches still rely on automatic differentiation to compute the sampled derivatives or projected vector products. A separate line of work avoids automatic differentiation entirely when computing derivatives. Finite differences approximate derivatives from

[†]SKLMS & Institute of Computational Mathematics and Scientific/Engineering Computing, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China.

[*]Corresponding author. (tzhou@lsec.cc.ac.cn).

nearby network evaluations [6,7], whereas randomized smoothing uses Stein's identity to express derivatives of a smoothed network as expectations of perturbed function values [10,27]. Although these methods reduce computational costs, their performance depends on the choice of method-specific parameters. Finite differences require a step size that balances truncation and round-off errors, randomized smoothing depends on both the smoothing bandwidth and the Monte Carlo sample size, and the randomized estimators require a sampling distribution and budget. For the stochastic methods, these choices directly affect estimator variance: insufficient or poorly matched sampling can yield high-variance residual estimates, while a small smoothing bandwidth can sharply amplify variance for high-order derivatives and may destabilize PINN training [10, 12, 25].

The stochastic Taylor derivative estimator (STDE) evaluates derivative-tensor contractions through univariate Taylor-mode automatic differentiation and uses them to construct stochastic estimators of linear differential operators [24]. STDE++ also studies exact recovery of mixed partial derivatives, reviewing the directional Taylor construction of Griewank, Utke, and Walther [8] and developing constructions based on higher-order Taylor expansions along nonlinear curves [25].

Within the Taylor-mode framework used by STDE [24], we use Waring decompositions of polynomial symbols to construct directional representations of homogeneous constant-coefficient differential operators. We call this construction the Waring decomposition of derivatives (WDD) method and specialize it to mixed partial derivatives. The resulting power-sum identities can reduce the number of Taylor coefficient evaluations. Compared with the STDE++ constructions used in our PDE experiments, WDD recovers the required mixed partials from directional Taylor coefficients at their derivative orders. Our main contributions are:

- We use power-sum identities to represent homogeneous linear combinations of partial derivatives by directional derivatives of the same order.
- For a single mixed partial, we give explicit coefficients and directions from the classical roots-of-unity decomposition and establish their minimum representation length using the monomial Waring rank. A batched Taylor-mode implementation evaluates the resulting directional Taylor coefficients independently and in parallel.
- We compare the WDD method with the deterministic directional Taylor construction reviewed in STDE++ on individual high-order partial derivatives, examining runtime across derivative patterns. KdV and KP-II PINN comparisons measure training time and solution error at equal update counts.

The remainder of the paper is organized as follows. Section 2 introduces the PINN formulation. Section 3 presents the derivative representation, constructs coefficients and directions of minimum length for a mixed partial, and describes their Taylor-mode evaluation. Section 4 evaluates the derivative backend on individual high-order partial derivatives and examines its use in complete KdV and KP-II PINN training. Section 5 concludes. Appendix A gives the coefficient comparisons underlying the representation and the roots-of-unity construction.

2. Preliminaries. Let $\Omega \subset \mathbb{R}^d$ be a computational domain and $\Gamma \subseteq \partial\Omega$ the part of its boundary on which constraints are prescribed. In the numerical examples, Ω is the space–time domain, d includes the time coordinate, and Γ includes the initial surface and the prescribed spatial boundary surfaces. We consider the scalar problem

$$\mathcal{A}u_\star = f \quad \text{in } \Omega, \qquad \mathcal{B}u_\star = g \quad \text{on } \Gamma, \tag{1}$$

where $\mathcal{A}$ is a differential operator and $\mathcal{B}$ is a boundary operator or a collection of boundary operators. The prescribed data are the forcing f on the computational domain and the constraint values g on the constrained boundary. In the PINN framework, the solution $u_\star$ is approximated by a scalar-valued neural network u_θ [15, 17, 23, 26]. Let $\{\boldsymbol{x}_{r,i}\}_{i=1}^{n_r} \subset \Omega$ and $\{\boldsymbol{x}_{b,i}\}_{i=1}^{n_b} \subset \Gamma$ denote the residual and constraint training point sets. Both sample counts are positive integers. For weights $\lambda_r, \lambda_b > 0$, the network parameters are obtained by minimizing

$$\mathcal{J}(\theta) := \frac{\lambda_r}{n_r} \sum_{i=1}^{n_r} \left|\mathcal{A}u_\theta(\boldsymbol{x}_{r,i}) - f(\boldsymbol{x}_{r,i})\right|^2 + \frac{\lambda_b}{n_b} \sum_{i=1}^{n_b} \left|\mathcal{B}u_\theta(\boldsymbol{x}_{b,i}) - g(\boldsymbol{x}_{b,i})\right|^2. \tag{2}$$

Evaluating the PDE residual in (2) requires derivatives of u_θ. For high-order equations, nested automatic differentiation can dominate the time and memory required for residual evaluation because it retains large intermediate computational graphs and repeatedly differentiates through the network.

3. Directional representations of derivatives. Let $\mathbb{N}_0$ denote the nonnegative integers and ∂_j denote differentiation with respect to the input coordinate x_j. For a multi-index $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_d) \in \mathbb{N}_0^d$, we define its order, factorial, and differential operator as

$$|\alpha| := \sum_{j=1}^{d} \alpha_j, \qquad \alpha! := \prod_{j=1}^{d} \alpha_j!, \qquad \partial^\alpha := \partial_1^{\alpha_1} \partial_2^{\alpha_2} \cdots \partial_d^{\alpha_d}. \tag{3}$$

For a direction $\boldsymbol{v} = (v_1, v_2, \ldots, v_d)^\top \in \mathbb{C}^d$, set $\boldsymbol{v}^\alpha := v_1^{\alpha_1} v_2^{\alpha_2} \cdots v_d^{\alpha_d}$. For the zero multi-index $\mathbf{0}$, we write $\partial^{\mathbf{0}} u = u$. Our method is based on the following proposition, which is a direct consequence of [5, Sec. 2].

PROPOSITION 3.1 (Directional representation). *Assume that $u : \mathbb{C}^d \to \mathbb{C}$ is holomorphic in a neighborhood of a given point $\boldsymbol{x} \in \mathbb{C}^d$. Let p and R be positive integers, and let $A_\alpha \in \mathbb{C}$ for $\alpha \in \mathbb{N}_0^d$ with $|\alpha| = p$. If $c_i \in \mathbb{C}$ and $\boldsymbol{v}_i \in \mathbb{C}^d$, $i = 1, 2, \ldots, R$, satisfy*

$$\sum_{|\alpha|=p} A_\alpha \boldsymbol{z}^\alpha = \frac{1}{p!} \sum_{i=1}^{R} c_i (\boldsymbol{v}_i^\top \boldsymbol{z})^p \quad \text{for all } \boldsymbol{z} \in \mathbb{C}^d, \tag{4}$$

then the following identity holds:

$$\sum_{|\alpha|=p} A_\alpha \partial^\alpha u(\boldsymbol{x}) = \frac{1}{p!} \sum_{i=1}^{R} c_i \left.\frac{d^p}{ds^p} u(\boldsymbol{x} + s\boldsymbol{v}_i)\right|_{s=0}. \tag{5}$$

Proposition 3.1 shows that once we find $c_i \in \mathbb{C}$ and $\boldsymbol{v}_i \in \mathbb{C}^d$, $i \in \{1, 2, \ldots, R\}$ satisfying (4), the p-th order derivative $\sum_{|\alpha|=p} A_\alpha \partial^\alpha u(\boldsymbol{x})$ can be written as a linear combination of p-th order directional derivatives. The latter are considerably cheaper to evaluate than the former when computed via Taylor-mode automatic differentiation [24]. The remaining key issue is how to find such c_i and $\boldsymbol{v}_i$.

We give two constructions in case of $\sum_{|\alpha|=p} A_\alpha \partial^\alpha = \partial^\nu$. The first construction follows from the inclusion–exclusion formula reviewed in [25, Sec. 6.1, Eq. (25)]:

$$c_i = (-1)^{p-|\gamma_i|} \prod_{j=1}^{d} \binom{\nu_j}{\gamma_{i,j}}, \quad \boldsymbol{v}_i = (\gamma_{i,1}, \ldots, \gamma_{i,d})^\top, \quad i = 1, 2, \ldots, R,$$

where $\gamma_1, \dots, \gamma_R$ enumerate the set

$$\big\{\gamma \in \mathbb{N}_0^d : \mathbf{0} \neq \gamma \leq \nu\big\}, \quad R = \prod_{j=1}^{d} (\nu_j + 1) - 1.$$

Here $\gamma \leq \nu$ means $\gamma_j \leq \nu_j$ for all $j = 1, 2, \dots, d$. The resulting value of R is not minimal for $p \geq 2$; see Corollary 3.2.

The second construction is the Waring decomposition from [4, Sec. 2, Eq. (8)]. Without loss of generality, we assume that the n nonzero entries of ν satisfy $1 \leq \nu_1 \leq \cdots \leq \nu_n$ and that $\nu_j = 0$ for $j = n+1, \dots, d$. Then (4), and hence (5), holds with

$$c_i = \frac{\nu!}{R} \prod_{j=2}^{n} \zeta_{i,j}, \quad \boldsymbol{v}_i = (1, \zeta_{i,2}, \dots, \zeta_{i,n}, 0, \dots, 0)^\top, \quad R = \prod_{j=2}^{n} (\nu_j + 1) \tag{6}$$

for $i = 1, 2, \dots, R$, where $(\zeta_{i,2}, \dots, \zeta_{i,n})$, $i = 1, \dots, R$, enumerate $U_{\nu_2+1} \times \cdots \times U_{\nu_n+1}$, with U_m defined by

$$U_m := \big\{\exp(2\pi \mathrm{i} k/m) : k = 0, 1, \dots, m-1\big\}, \quad m = 1, 2, \dots. \tag{7}$$

Corollary 3.2 (Minimum representation length). *Let $\nu \in \mathbb{N}_0^d$ satisfy $|\nu| = p \geq 1$ and have n nonzero entries, ordered as $1 \leq \nu_1 \leq \cdots \leq \nu_n$ after a coordinate permutation. In (4), set $A_\nu = 1$ and $A_\alpha = 0$ for $\alpha \neq \nu$. Among all choices of a positive integer R, coefficients $c_i \in \mathbb{C}$ and directions $\boldsymbol{v}_i \in \mathbb{C}^d$, $i = 1, \dots, R$, satisfying (4), the minimum value of R is*

$$R_\nu := \prod_{j=2}^{n} (\nu_j + 1). \tag{8}$$

The construction (6)–(7) attains this minimum.

Proof. In this case, (4) is a Waring decomposition of $\boldsymbol{z}^\nu$ with coefficients $c_i/p!$ and at most R nonzero terms. The classical monomial rank formula [5, Cor. 3.3 and Remark 2.3] gives $R \geq R_\nu$. The construction (6)–(7) realizes $R = R_\nu$ by (30), so the bound is attained. ∎

Testing (5) on degree-p monomials gives (4), so R_ν is also the minimum length of representations of $\partial^\nu u$ with fixed coefficients and directions, valid for every holomorphic u.

Taylor-mode automatic differentiation [24] computes the coefficient of s^p in each $u(\boldsymbol{x} + s\boldsymbol{v}_i)$, and (5) gives $\partial^\nu u(\boldsymbol{x})$ by summing these coefficients with weights c_i. We restore the original coordinate positions in each $\boldsymbol{v}_i$, precompute the weights and directions, and batch the Taylor propagations. For a fixed network architecture and evaluation batch size, an R-term representation costs $O(Rp^2)$ before coefficient reuse, excluding decomposition preparation [24, Sec. 4.2].

Open question: for a general differential operator, how can one find coefficients c_i and directions $\boldsymbol{v}_i$ satisfying

$$\sum_{|\alpha| \leq p} A_\alpha \partial^\alpha u(\boldsymbol{x}) = \frac{1}{p!} \sum_{i=1}^{R} c_i \, \frac{d^p}{ds^p} u(\boldsymbol{x} + s\boldsymbol{v}_i)\Big|_{s=0}$$

with the minimum possible R?

Related reference: V. Beorchia and F. Galuppi, *Generic identifiability of pairs of ternary forms*, Collectanea Mathematica, 74 (2023), pp. 529–545 [2].

4. Numerical experiments. We evaluate WDD in two complementary settings. First, we measure the evaluation time of individual high-order partial derivatives across various orders and multi-index patterns, using the construction (6)–(7), whose minimum length is established in Corollary 3.2. Second, we assess PINN training on three nonlinear differential equations: a one-dimensional KdV equation with a gradient-enhanced objective, a two-dimensional KdV equation with mixed derivatives, and a KP-II equation. For the PDE problems, we evaluate real power-sum representations by (5), reusing Taylor coefficients across derivative terms. The compared methods compute the same mathematical derivatives. The PDE experiments compare WDD with the STDE++ Taylor constructions under identical loss objectives and matched computational conditions.

For PDE residuals, WDD uses power-sum identities to recover the required mixed partials at their derivative orders, avoiding the higher-order expansions used by the compared STDE++ constructions. For an individual mixed partial, the roots-of-unity construction attains the minimum representation length in Corollary 3.2. Conjugate reuse can further reduce the number of Taylor coefficient evaluations.

4.1. Individual high-order partial derivatives. We compare WDD with a centered-polarization implementation of the directional Taylor identity reviewed in STDE++ [25, Section 6.1, Eq. (25)] for individual partial derivatives $\partial_{i_1}\partial_{i_2}\cdots\partial_{i_p}u$. We set the batch size to $B = 1000$. The test network has 16 inputs and three hidden layers of width 128 with tanh activations, and both methods use identical weights and inputs and the same Taylor propagation routine on a Tesla V100. Here $R = R_\nu$ is given by (8). For the real-valued networks and real inputs used here, we reuse Taylor coefficients at conjugate directions.

The order-p coefficient counts in Table 4.1 are per input point, after conjugate reuse in WDD and merging redundant directions in STDE++. Times are the mean and sample standard deviation of three seed medians (30 synchronized calls after 10 warm-ups per seed), excluding direction preparation and parameter backpropagation.

TABLE 4.1

Ranks, Taylor coefficient counts, and evaluation times on a Tesla V100 for WDD and STDE++ [25, Section 6.1]. Times are means and sample standard deviations of three seed medians, and speedups are means of paired ratios of STDE++ to WDD times.

Target	p	R	Taylor coefficients		Time (ms)		Speedup
			STDE++	WDD	STDE++	WDD	
$\partial_1\partial_2\partial_3\partial_4\partial_5\partial_6 u$	6	32	32	32	29.605 ± 0.186	29.600 ± 0.145	1.00×
$\partial_1^2\partial_2^2\partial_3^2\partial_4^2 u$	8	27	40	14	60.533 ± 0.028	45.456 ± 0.029	1.33×
$\partial_1^3\partial_2^3\partial_3^3 u$	9	16	28	10	51.341 ± 0.007	39.932 ± 0.019	1.29×
$\partial_1^4\partial_2^4\partial_3^4 u$	12	25	49	13	153.037 ± 0.025	85.421 ± 0.009	1.79×
$\partial_1^5\partial_2^5\partial_3^5 u$	15	36	100	20	479.972 ± 0.006	195.631 ± 0.026	2.45×

4.2. PINN training on nonlinear differential equations. For all PDE benchmarks, u_θ is a fully connected network with four hidden layers of width 128, tanh activations, and a scalar linear output. We train with Adam [16] using a learning rate of 10^{-5} for the KdV problems and 10^{-4} for KP-II.

At each update we independently resample 400 interior space–time points and points on each initial and spatial boundary face: 128 points per face in 1D and

256 per face in 2D. Let $\mathcal{T}_r$, $\mathcal{T}_0$, and $\mathcal{T}_b$ be the interior, initial, and combined spatial boundary point sets. Conditions on the same face use the same points. The residual and initial–boundary penalty are defined separately for each benchmark below.

For each of five seeds, the paired methods share the initialization, loss weights, and samples at each common update index. Both methods are trained for 25,000 updates for one-dimensional KdV and 20,000 updates for two-dimensional KdV and KP-II. The KdV averages use seeds 20260919–20260923, and the KP-II averages use seeds 20260936–20260940.

For a test set D of space–time points with nonzero reference norm, we define the relative L^2 error by

$$\mathrm{RE} := \frac{\left(\sum_{\boldsymbol{x}\in D} |u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2\right)^{1/2}}{\left(\sum_{\boldsymbol{x}\in D} |u_\star(\boldsymbol{x})|^2\right)^{1/2}}, \tag{9}$$

evaluated on 10,000 uniform space–time test points. The test set is chosen once, independently of training, and shared by paired methods.

Tables report training times and RE as means and sample standard deviations across seeds. Training speedups are means of paired ratios of STDE++ to WDD training times. Figure curves show mean errors against mean training times at common update indices, with shading extending one sample standard deviation upward.

Polynomial identities. In the polynomial identities below, coordinate letters also denote the corresponding formal variables.

4.2.1. Gradient-enhanced PINN for the KdV equation. We consider the one-dimensional KdV equation

$$\begin{cases} u_t + uu_x + 0.0025u_{xxx} = 0, & x \in (-1,1), \quad t \in (0,1), \\ u(x,0) = u_\star(x,0), & x \in [-1,1], \\ u(-1,t) = u_\star(-1,t), & t \in (0,1), \\ u(1,t) = u_\star(1,t), \quad \partial_x u(1,t) = \partial_x u_\star(1,t), & t \in (0,1). \end{cases} \tag{10}$$

We use the traveling solitary-wave reference solution

$$u_\star(x,t) = 0.75\ \mathrm{sech}^2\big(5(x + 0.5 - 0.25t)\big). \tag{11}$$

We define the PDE residual by

$$r_\theta(x,t) := (u_\theta)_t + u_\theta(u_\theta)_x + 0.0025(u_\theta)_{xxx},$$

and define the initial and boundary penalty by

$$\begin{aligned} \mathcal{J}_b(\theta) = {} & \frac{1}{|\mathcal{T}_0|} \sum_{(x,0)\in\mathcal{T}_0} \big[u_\theta(x,0) - u_\star(x,0)\big]^2 + \frac{1}{128} \sum_{(-1,t)\in\mathcal{T}_b} \big[u_\theta(-1,t) - u_\star(-1,t)\big]^2 \\ & + \frac{1}{128} \sum_{(1,t)\in\mathcal{T}_b} \Big\{ \big[u_\theta(1,t) - u_\star(1,t)\big]^2 + \big[\partial_x u_\theta(1,t) - \partial_x u_\star(1,t)\big]^2 \Big\}. \end{aligned}$$

The objective is

$$\mathcal{J}(\theta) = \frac{1}{|\mathcal{T}_r|} \sum_{(x,t)\in\mathcal{T}_r} \Big[r_\theta^2 + \lambda_1 \big((\partial_x r_\theta)^2 + (\partial_t r_\theta)^2 \big) \Big] + \lambda_2 \mathcal{J}_b(\theta), \tag{12}$$

where $\lambda_1 = 10^{-3}$ and $\lambda_2 = 10$ are the loss weights. Every residual and residual derivative in the interior sum is evaluated at the sampled point.

Let $\boldsymbol{x} = (x, t)^\top$ be a network input point. We express the mixed symbol used to evaluate u_{xxxt}, namely $x^3 t$, as

$$x^3 t = \tfrac{1}{6}\big[(x+t)^4 - (x-t)^4\big] - \tfrac{1}{48}\big[(x+2t)^4 - (x-2t)^4\big]. \tag{13}$$

By Proposition 3.1 with (13) and the pure symbol x^4, we have

$$\begin{aligned} u_{xxxx} &= \frac{d^4}{ds^4} u(x+s,t)\bigg|_{s=0}, \\ u_{xxxt} &= \frac{1}{6}\,\frac{d^4}{ds^4}\big[u(x+s,t+s) - u(x+s,t-s)\big]\bigg|_{s=0} \\ &\quad - \frac{1}{48}\,\frac{d^4}{ds^4}\big[u(x+s,t+2s) - u(x+s,t-2s)\big]\bigg|_{s=0}. \end{aligned} \tag{14}$$

These five directions also provide the lower-order coefficients required by the objective. The STDE++ baseline uses two Taylor expansions through order seven and one through order two, following the gradient-enhanced construction in [25, Appendix I.4]. Table 4.2 and Figure 4.1 show that the WDD method attains comparable mean final errors with a 1.44× training speedup relative to STDE++.

TABLE 4.2

Numerical results for the one-dimensional KdV equation (10) *after 25,000 updates, reported as means and sample standard deviations over five seeds.*

Method	Training time (s)	RE
WDD	1123.8 ± 15.7	$4.955 \times 10^{-3} \pm 1.297 \times 10^{-3}$
STDE++	1614.9 ± 13.4	$4.954 \times 10^{-3} \pm 1.298 \times 10^{-3}$

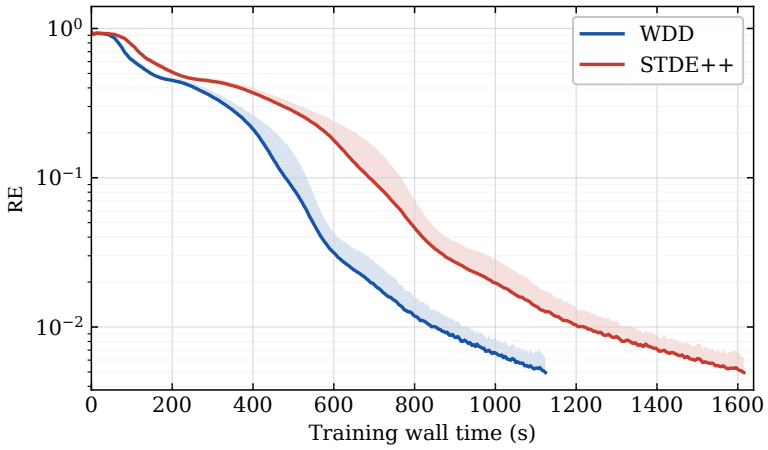


FIG. 4.1. RE *during training for the one-dimensional KdV equation* (10). *Curves show mean errors against mean training times over five seeds at common update indices; shading extends one sample standard deviation upward.*

4.2.2. Mixed-derivative PINN for a two-dimensional KdV equation. We consider the KdV-type equation in [24, Eq. (20)],

$$\begin{cases} u_{ty} + u_{xxxy} + 3(u_x u_y)_x - u_{xx} + 2u_{yy} = 0, & (x,y) \in (-1,1)^2, \quad t \in (0,1), \\ u(x,y,0) = u_\star(x,y,0), & (x,y) \in [-1,1]^2, \\ u(-1,y,t) = u_\star(-1,y,t), & y \in [-1,1], \quad t \in (0,1), \\ \quad u(1,y,t) = u_\star(1,y,t), & \\ \partial_x u(1,y,t) = \partial_x u_\star(1,y,t), & y \in [-1,1], \quad t \in (0,1), \\ \quad u(x,-1,t) = u_\star(x,-1,t), & \\ \partial_y u(x,-1,t) = \partial_y u_\star(x,-1,t), & x \in [-1,1], \quad t \in (0,1). \end{cases} \tag{15}$$

We use the oblique-kink reference solution

$$u_\star(x,y,t) = \tanh\big((x+y)/2 - t\big). \tag{16}$$

Only the PDE is taken from the cited work; the domain, exact solution, and initial–boundary conditions specify the present benchmark.

We define the PDE residual by

$$r_\theta(x,y,t) := (u_\theta)_{ty} + (u_\theta)_{xxxy} + 3\partial_x\big[(u_\theta)_x (u_\theta)_y\big] - (u_\theta)_{xx} + 2(u_\theta)_{yy},$$

and define the initial and boundary penalty by

$$\begin{aligned} \mathcal{J}_b(\theta) = {} & \frac{1}{|\mathcal{T}_0|} \sum_{(x,y,0)\in\mathcal{T}_0} \big[u_\theta(x,y,0) - u_\star(x,y,0)\big]^2 \\ & + \frac{1}{256} \sum_{(-1,y,t)\in\mathcal{T}_b} \big[u_\theta(-1,y,t) - u_\star(-1,y,t)\big]^2 \\ & + \frac{1}{256} \sum_{(1,y,t)\in\mathcal{T}_b} \Big\{ \big|(u_\theta - u_\star)(1,y,t)\big|^2 + \big|\partial_x(u_\theta - u_\star)(1,y,t)\big|^2 \Big\} \\ & + \frac{1}{256} \sum_{(x,-1,t)\in\mathcal{T}_b} \Big\{ \big|(u_\theta - u_\star)(x,-1,t)\big|^2 + \big|\partial_y(u_\theta - u_\star)(x,-1,t)\big|^2 \Big\}. \end{aligned}$$

The objective is

$$\mathcal{J}_{2\mathrm{D}}(\theta) = \frac{1}{|\mathcal{T}_r|} \sum_{(x,y,t)\in\mathcal{T}_r} r_\theta(x,y,t)^2 + \lambda_2 \mathcal{J}_b(\theta). \tag{17}$$

The boundary weight is the value specified in Section 4.2.1.

We express the additional time–space identity used with (13) after replacing t by y as

$$yt = \tfrac{1}{4}\big[(y+t)^2 - (y-t)^2\big], \tag{18}$$

where the coordinate letters are the formal variables introduced in the common protocol. Let $\boldsymbol{x} = (x,y,t)^\top$ be the network input point. By Proposition 3.1 with (13)

and (18), we have

$$u_{xxxy} = \frac{1}{6}\frac{d^4}{ds^4}\big[u(x+s,y+s,t) - u(x+s,y-s,t)\big]\Big|_{s=0} - \frac{1}{48}\frac{d^4}{ds^4}\big[u(x+s,y+2s,t) - u(x+s,y-2s,t)\big]\Big|_{s=0}, \qquad (19)$$
$$u_{ty} = \frac{1}{4}\frac{d^2}{ds^2}\big[u(x,y+s,t+s) - u(x,y+s,t-s)\big]\Big|_{s=0}.$$

The reconstruction (19) requires Taylor expansions along six real directions. The lower-order Taylor coefficients also recover $u_x, u_y, u_{xx}, u_{xy}, u_{yy}$.

The STDE++ baseline uses one order-nine and two order-three Taylor expansions, following the lower-order alternative in [25, Appendix I.4]. The results in Table 4.3 and Figure 4.2 show that the WDD method attains comparable mean final errors with a 1.62× training speedup relative to STDE++.

TABLE 4.3
Numerical results for the two-dimensional KdV equation (15) *after 20,000 updates, reported as means and sample standard deviations over five seeds.*

Method	Training time (s)	RE
WDD	1217.1 ± 10.9	$1.020 \times 10^{-3} \pm 2.598 \times 10^{-4}$
STDE++	1972.6 ± 35.5	$1.020 \times 10^{-3} \pm 2.598 \times 10^{-4}$

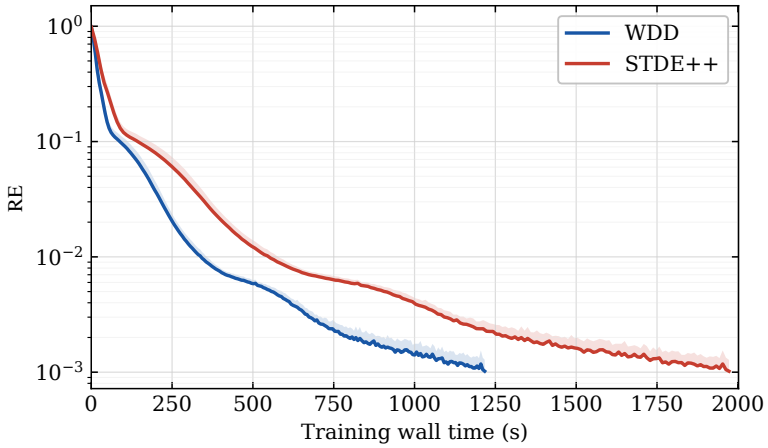


FIG. 4.2. RE *during training for the two-dimensional KdV equation* (15). *Curves show mean errors against mean training times over five seeds at common update indices; shading extends one sample standard deviation upward.*

4.2.3. KP-II equation. We consider the KP-II line-soliton problem in [22, Eqs. (3.7)–(3.9)]. With a slight abuse of notation in this subsection, $\Omega = (-3,3)^2$ denotes the spatial part of the computational domain. The initial–boundary value

problem is

$$\begin{cases} u_{tx} + 6(u_x^2 + uu_{xx}) + u_{xxxx} + 3u_{yy} = 0, & (x,y) \in \Omega, \quad t \in (-\frac{1}{2}, \frac{1}{2}), \\ u(x,y,-\frac{1}{2}) = u_\star(x,y,-\frac{1}{2}), & (x,y) \in \overline{\Omega}, \\ u(x,y,t) = u_\star(x,y,t), & (x,y) \in \partial\Omega, \quad t \in (-\frac{1}{2}, \frac{1}{2}). \end{cases} \tag{20}$$

We use the reference solution

$$u_\star(x,y,t) = \frac{18e^{3x+6y-63t}}{(1+e^{3x+6y-63t})^2}. \tag{21}$$

We define the PDE residual by

$$r_\theta := (u_\theta)_{tx} + 6\big[(u_\theta)_x^2 + u_\theta(u_\theta)_{xx}\big] + (u_\theta)_{xxxx} + 3(u_\theta)_{yy}.$$

Let $\mathcal{F}$ be the set of four spatial boundary faces. The objective is

$$\mathcal{J}_{\mathrm{KP}}(\theta) = \frac{1}{|\mathcal{T}_r|}\sum_{\boldsymbol{x}\in\mathcal{T}_r} r_\theta(\boldsymbol{x})^2 + \lambda_2\Bigg\{\frac{1}{|\mathcal{T}_0|}\sum_{\boldsymbol{x}\in\mathcal{T}_0}|u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2 + \sum_{F\in\mathcal{F}}\frac{1}{256}\sum_{\substack{\boldsymbol{x}\in\mathcal{T}_b\\(x,y)\in F}}|u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2\Bigg\}, \qquad \lambda_2 = 10. \tag{22}$$

Let $\boldsymbol{x} = (x,y,t)^\top$ be the network input point. By Proposition 3.1 with the power-sum identity $xt = (1/4)[(x+t)^2 - (x-t)^2]$, we have

$$\begin{aligned} (u_\theta)_{xxxx} &= \frac{d^4}{ds^4}u_\theta(x+s,y,t)\Big|_{s=0}, \\ (u_\theta)_{tx} &= \frac{1}{4}\,\frac{d^2}{ds^2}\big[u_\theta(x+s,y,t+s) - u_\theta(x+s,y,t-s)\big]\Big|_{s=0}, \\ (u_\theta)_{yy} &= \frac{d^2}{ds^2}u_\theta(x,y+s,t)\Big|_{s=0}. \end{aligned} \tag{23}$$

The order-four expansion also supplies u_θ, $(u_\theta)_x$, and $(u_\theta)_{xx}$. The STDE++ baseline recovers $(u_\theta)_{tx}$ from the fifth-order expansion along $\boldsymbol{x}+s^2(1,0,0)^\top/2+s^3(0,0,1)^\top/6$, together with coordinate Taylor expansions for the remaining terms, following its published implementation [25]. Table 4.4 and Figure 4.3 show final mean errors of 3.726×10^{-2} and 4.052×10^{-2} for WDD and STDE++, respectively, with a $1.32\times$ training speedup.

TABLE 4.4

Numerical results for the KP-II equation (20) *after 20,000 updates, reported as means and sample standard deviations over five seeds.*

Method	Training time (s)	RE
WDD	913.1 ± 37.3	$3.726\times10^{-2} \pm 6.034\times10^{-3}$
STDE++	1205.6 ± 29.5	$4.052\times10^{-2} \pm 7.050\times10^{-3}$

Across the PDE experiments, the WDD method attains comparable mean final errors at equal update counts. It requires less training time, with both interior and constraint training points resampled at every update. Table 4.5 reports the peak allocated GPU memory under the measurement protocol stated in its caption.

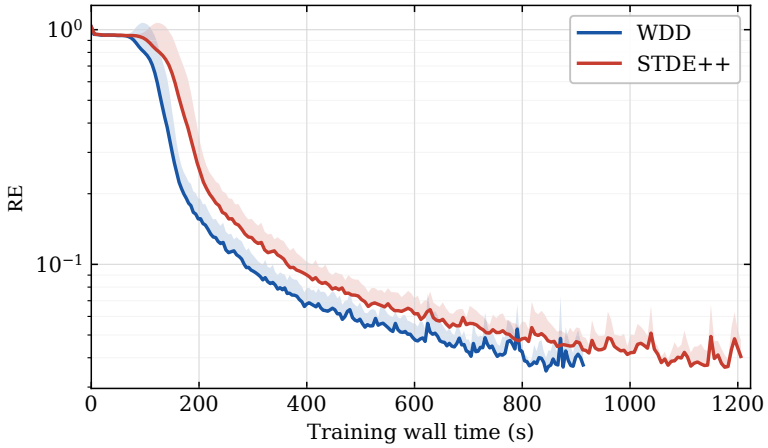


FIG. 4.3. RE *during training for the KP-II equation* (20). *Curves show mean errors against mean training times over five seeds at common update indices; shading extends one sample standard deviation upward.*

TABLE 4.5

Peak allocated GPU memory (MiB) in 30-s measurements after ten warm-up calls or updates. Entries are maxima over three seeds for individual partials and five seeds for PDE training; the latter includes loss evaluation, backpropagation, and Adam updates.

Target	STDE++	WDD
Individual partial derivatives		
$\partial_1\partial_2\partial_3\partial_4\partial_5\partial_6 u$	725.2	725.2
$\partial_1^2\partial_2^2\partial_3^2\partial_4^2 u$	1172.9	834.4
$\partial_1^3\partial_2^3\partial_3^3 u$	929.7	671.8
$\partial_1^4\partial_2^4\partial_3^4 u$	2095.6	1129.5
$\partial_1^5\partial_2^5\partial_3^5 u$	5260.4	2131.7
PDE training		
One-dimensional KdV	153.1	163.5
Two-dimensional KdV	152.4	183.7
KP-II	126.0	109.6

5. Conclusion. Within the Taylor-mode framework used by STDE [24], WDD constructs exact directional representations of high-order derivatives from polynomial symbols. The roots-of-unity construction supplies explicit coefficients and complex directions for a mixed partial, and classical monomial Waring-rank theory establishes the minimum representation length. The WDD method evaluates these representations by batched Taylor propagation, while real power-sum identities allow Taylor coefficients to be reused across PDE derivative terms. The experiments show faster individual-partial evaluation for several tested patterns and shorter PINN training time than STDE++ at equal update counts. The WDD method attains comparable mean final errors for the tested KdV and KP-II problems.

Appendix A. Algebraic verification.

This appendix proves Proposition 3.1 by coefficient comparison and verifies the explicit decomposition in (30).

Proof of Proposition 3.1. For $\boldsymbol{v} \in \mathbb{C}^d$, the multivariate Taylor formula [19, Sec. 3.17] gives

$$\frac{d^p}{ds^p} u(\boldsymbol{x} + s\boldsymbol{v})\bigg|_{s=0} = \sum_{|\alpha|=p} \frac{p!}{\alpha!} \boldsymbol{v}^\alpha \partial^\alpha u(\boldsymbol{x}). \tag{24}$$

In this proof, write the normalized directional coefficient as

$$T_p(\boldsymbol{x}; \boldsymbol{v}) := \frac{1}{p!} \frac{d^p}{ds^p} u(\boldsymbol{x} + s\boldsymbol{v})\bigg|_{s=0}. \tag{25}$$

Combining (24) and (25), we obtain

$$\sum_{i=1}^{R} c_i T_p(\boldsymbol{x}; \boldsymbol{v}_i) = \sum_{|\alpha|=p} \left(\frac{1}{\alpha!} \sum_{i=1}^{R} c_i \boldsymbol{v}_i^\alpha \right) \partial^\alpha u(\boldsymbol{x}). \tag{26}$$

Thus, (5) holds whenever

$$\sum_{i=1}^{R} c_i \boldsymbol{v}_i^\alpha = \alpha!\, A_\alpha, \qquad \alpha \in \mathbb{N}_0^d, \quad |\alpha| = p. \tag{27}$$

By the multinomial theorem,

$$\frac{1}{p!} \sum_{i=1}^{R} c_i (\boldsymbol{v}_i^\top \boldsymbol{z})^p = \sum_{|\alpha|=p} \left(\frac{1}{\alpha!} \sum_{i=1}^{R} c_i \boldsymbol{v}_i^\alpha \right) \boldsymbol{z}^\alpha. \tag{28}$$

Comparing coefficients in (4) using (28) gives (27), and hence (5). ∎

LEMMA A.1 (Roots-of-unity filter). *For every integer $m \geq 1$ and every integer k, the following identity holds:*

$$\sum_{\zeta \in U_m} \zeta^k = \begin{cases} m, & m \mid k, \\ 0, & m \nmid k. \end{cases} \tag{29}$$

Proof. Set $\omega := \exp(2\pi\mathrm{i}/m)$. By the definition of U_m, every element of U_m has the form ω^ℓ for exactly one $\ell \in \{0, 1, \ldots, m-1\}$. We express the root sum in (29) as

$$\sum_{\zeta \in U_m} \zeta^k = \sum_{\ell=0}^{m-1} \left(\omega^k\right)^\ell.$$

The divisibility condition $m \mid k$ implies that $\omega^k = 1$ and the root sum in (29) equals m.

The nondivisibility condition $m \nmid k$ implies that $\omega^k \neq 1$. By the finite geometric-series formula, we have

$$\sum_{\ell=0}^{m-1} \left(\omega^k\right)^\ell = \frac{1-\omega^{km}}{1-\omega^k} = 0,$$

where the equality to zero follows from $\omega^m = 1$. ∎

Using (6)–(7), the decomposition of Buczyńska, Buczyński, and Teitler [4, Sec. 2, Eq. (8)] takes the form

$$p!\, \boldsymbol{z}^\nu = \sum_{i=1}^{R_\nu} c_i \left(z_1 + \sum_{j=2}^{n} \zeta_{i,j} z_j \right)^p. \tag{30}$$

Verification of (30). We now prove (30) by coefficient comparison. Let $\alpha = (\alpha_1, \ldots, \alpha_n) \in \mathbb{N}_0^n$ be an arbitrary multi-index with $|\alpha| = p$. By (6) and the enumeration of root tuples, the coefficient of $\boldsymbol{z}^\alpha$ on the right-hand side of (30) is

$$\frac{\nu!}{\prod_{j=2}^n (\nu_j + 1)} \binom{p}{\alpha} \prod_{j=2}^{n} \left(\sum_{\zeta_j \in U_{\nu_j+1}} \zeta_j^{\alpha_j+1} \right). \tag{31}$$

We now show that a nonzero product of root sums in (31) forces $\alpha = \nu$. Lemma A.1 implies that the product of root sums in (31) can be nonzero only if $\alpha_j + 1$ is a positive integer multiple of $\nu_j + 1$ for every $j = 2, \ldots, n$. By the root-filter divisibility conditions from Lemma A.1, we know that each α_j is either ν_j or at least $2\nu_j + 1$. We now prove the exclusion of the case $\alpha_j \geq 2\nu_j + 1$ for some $j \geq 2$ in the root-sum index range of (31). By the divisibility conditions for the root sums in (31), we know that $\alpha_k \geq \nu_k$ for every $k = 2, \ldots, n$. By the total-degree identities and the ordering of the active exponents, we have

$$\alpha_1 = p - \sum_{k=2}^{n} \alpha_k \leq \nu_1 - \nu_j - 1 < 0,$$

where the contradiction with the negative upper bound follows from $\alpha_1 \geq 0$. By the contradiction with a negative first exponent and the divisibility alternatives from Lemma A.1, we know that $\alpha_j = \nu_j$ for $j = 2, \ldots, n$. By the degree condition $|\alpha| = p$, we have $\alpha_1 = \nu_1$.

The coefficient formula (31) implies that the coefficient vanishes when the product of root sums is zero. By the zero-product case and the exponent equalities in the nonzero-product case of (31), we know that the coefficient in (31) vanishes unless $\alpha_j = \nu_j$ for every $j = 1, \dots, n$. By (31) with $\alpha = \nu$, we have

$$\frac{\nu!}{\prod_{j=2}^{n}(\nu_j+1)} \frac{p!}{\nu!} \prod_{j=2}^{n}(\nu_j+1) = p!.$$

The vanishing coefficients and the coefficient value for $\boldsymbol{z}^{\nu}$ imply that (30) holds. ∎

REFERENCES

[1] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind, *Automatic differentiation in machine learning: a survey*, Journal of Machine Learning Research, 18 (2018), pp. 1–43.

[2] Valentina Beorchia and Francesco Galuppi, *Generic identifiability of pairs of ternary forms*, Collectanea Mathematica, 74 (2023), pp. 529–545.

[3] Jesse Bettencourt, Matthew J. Johnson, and David Duvenaud, *Taylor-mode automatic differentiation for higher-order derivatives in JAX*, in NeurIPS Workshop on Program Transformations for Machine Learning, 2019.

[4] Weronika Buczyńska, Jarosław Buczyński, and Zach Teitler, *Waring decompositions of monomials*, Journal of Algebra, 378 (2013), pp. 45–57.

[5] Enrico Carlini, Maria Virginia Catalisano, and Anthony V. Geramita, *The solution to the Waring problem for monomials and the sum of coprime monomials*, Journal of Algebra, 370 (2012), pp. 5–14.

[6] Pao-Hsiung Chiu, Jian Cheng Wong, Chinchun Ooi, My Ha Dao, and Yew-Soon Ong, *CAN-PINN: A fast physics-informed neural network based on coupled-automatic–numerical differentiation method*, Computer Methods in Applied Mechanics and Engineering, 395 (2022), p. 114909.

[7] Bengt Fornberg, *Generation of finite difference formulas on arbitrarily spaced grids*, Mathematics of Computation, 51 (1988), pp. 699–706.

[8] Andreas Griewank, Jean Utke, and Andrea Walther, *Evaluating higher derivative tensors by forward propagation of univariate Taylor series*, Mathematics of Computation, 69 (2000), pp. 1117–1130.

[9] Andreas Griewank and Andrea Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2nd ed., 2008.

[10] Di He, Shanda Li, Wenlei Shi, Xiaotian Gao, Jia Zhang, Jiang Bian, Liwei Wang, and Tie-Yan Liu, *Learning physics-informed neural networks without stacked back-propagation*, in Proceedings of the 26th International Conference on Artificial Intelligence and Statistics, vol. 206 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 3034–3047.

[11] Zheyuan Hu, Zekun Shi, George Em Karniadakis, and Kenji Kawaguchi, *Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 424 (2024), p. 116883.

[12] ———, *Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs*, Journal of Computational Physics, 521 (2025), p. 113524.

[13] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, and Kenji Kawaguchi, *Tackling the curse of dimensionality with physics-informed neural networks*, Neural Networks, 176 (2024), p. 106369.

[14] Michael F. Hutchinson, *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines*, Communications in Statistics: Simulation and Computation, 18 (1989), pp. 1059–1076.

[15] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, *Physics-informed machine learning*, Nature Reviews Physics, 3 (2021), pp. 422–440.

[16] Diederik P. Kingma and Jimmy Ba, *Adam: A method for stochastic optimization*, in International Conference on Learning Representations (ICLR), 2015.

[17] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, 9 (1998), pp. 987–1000.

[18] Ruichen Li, Haotian Ye, Du Jiang, Xuelan Wen, Chuwei Wang, Zhe Li, Xiang Li, Di He, Ji Chen, Weiluo Ren, and Liwei Wang, *A computational framework for neural network-based variational Monte Carlo with Forward Laplacian*, Nature Machine Intelligence, 6 (2024), pp. 209–219.

[19] Lynn H. Loomis and Shlomo Sternberg, *Advanced Calculus*, Jones and Bartlett, Boston, MA, revised ed., 1990.

[20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala, *PyTorch: An imperative style, high-performance deep learning library*, in Advances in Neural Information Processing Systems, vol. 32, 2019.

[21] Barak A. Pearlmutter, *Fast exact multiplication by the Hessian*, Neural Computation, 6 (1994), pp. 147–160.

[22] Juncai Pu and Yong Chen, *Lax pairs informed neural networks solving integrable systems*, arXiv preprint arXiv:2401.04982, (2024).

[23] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.

[24] Zekun Shi, Zheyuan Hu, Min Lin, and Kenji Kawaguchi, *Stochastic Taylor derivative estimator: Efficient amortization for arbitrary differential operators*, Advances in Neural Information Processing Systems (NeurIPS), 37 (2024), pp. 122316–122353.

[25] ———, *STDE++: Polynomial-time amortization for linear differential operators*, Journal of Machine Learning Research, 27 (2026), pp. 1–50.

[26] Justin Sirignano and Konstantinos Spiliopoulos, *DGM: A deep learning algorithm for solving partial differential equations*, Journal of Computational Physics, 375 (2018), pp. 1339–1364.

[27] Charles M. Stein, *Estimation of the mean of a multivariate normal distribution*, The Annals of Statistics, 9 (1981), pp. 1135–1151.